

Lab 05b · El cajón de las correspondencias

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

90 minutos · Spring Boot 4.1.0 · Java 25 (Temurin) · PostgreSQL 16 embebido

Resumen

Que cuando dos fichas se apuntan **las dos en plural** ya no basta una casilla: hace falta una tercera tabla que nadie escribe, y JPA la crea, la llena y la vacía sola. Y que el tipo de la colección decide el precio: **1 sentencia con Set y 6 con List** para exactamente el mismo cambio, contadas en pantalla.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	3
1.4. La puesta a punto	3
2. El caso	3
2.1. El cajón de las correspondencias, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · La tabla que aparece sola	3
3.2. Paso 2 · Meter y sacar papelitos	6
3.3. Paso 3 · El otro lado del cajón	7
3.4. Paso 4 · Set contra List, contado	9
3.5. Paso 5 · Un nombre de método que atraviesa el cajón	11
3.6. Paso 6 · Cuando @ManyToMany deja de servir	13
4. Lo que aprendiste	14
5. Para profundizar	15
6. Antes de cerrar	15

1 Antes de empezar

1.1 Qué vas a lograr

En el Lab 05 un trámite pertenecía a **un** contribuyente, y eso cabía en una casilla de la ficha. Hoy un trámite pide **varios** documentos y cada documento sirve a **varios** trámites. En los dos sentidos, plural. Eso no cabe en ninguna casilla de ninguna de las dos fichas.

Vas a declarar esa relación con dos anotaciones, vas a **mirar la tabla que aparece sola** cuando lo haces, y vas a **contar las sentencias** que cuesta adjuntar y quitar un documento según el tipo de colección que uses. Al final vas a ver **dónde se rompe** @ManyToMany y en qué se convierte cuando eso pasa.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
El Lab 05 hecho	Sabes qué es @ManyToOne y qué hace mappedBy	Es imprescindible aquí
Estar en la carpeta del lab	cd labs/lab-05b-muchos-a-muchos/practica	El cd no da error
Nada más	La base viaja dentro del repositorio	—

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa: el compilador ignora la sangría. Si una línea se parte, el editor te la marca. El código completo está en `labs/lab-05b-muchos-a-muchos/solucion/`.

1.4 La puesta a punto

```
cd labs/lab-05b-muchos-a-muchos/practica
./mvnw spring-boot:run
```

Escucha en el **8110** y su PostgreSQL en el **55447**. **Párala siempre con Ctrl+C**: los dos errores de arranque del Lab 04 —el puerto ocupado y el candado tomado— salen aquí igual, con los números de este lab.

2 El caso

Un trámite de inicio de actividades pide cédula, escritura y balance. Uno de cambio de representante pide cédula, escritura y poder. **La cédula la piden los dos**, y no hay dos cédulas: es el mismo tipo de documento.

2.1 El cajón de las correspondencias, que es la metáfora de este laboratorio

Un cajón lleno de papelitos, y en cada papelito dos números.

En el Lab 05 la ficha del trámite llevaba escrito el número de su contribuyente, en una casilla. Eso valía porque el contribuyente era **uno**.

Hoy no vale. Si la ficha del trámite llevara la lista de sus documentos, sería una lista que crece. Y si la ficha del documento llevara la lista de sus trámites, sería otra lista que crece más. Las dos fichas se llenarían de números.

Así que el archivador tiene **un tercer cajón**, y dentro sólo hay papelitos. Cada papelito dice exactamente dos cosas:

trámite nº 1 — documento nº 3

Nada más. Ni fecha, ni firma, ni nombre. **Adjuntar un documento es meter un papelito.**

Quitarlo es sacar ese papelito — y fíjate en lo que eso NO hace: la ficha del documento sigue en su cajón, intacta, porque el papelito no era el documento.

Ese cajón es la tabla `tramite_documento`. **Nadie la escribe**: la declara una anotación. No tiene clase, no tiene repositorio, y aun así tiene filas.

Y la pregunta que cierra el laboratorio, que es de archivista:

¿Y si hiciera falta anotar en el papelito la fecha en que se adjuntó, o quién lo trajo?

No cabe. Un papelito de este cajón tiene sitio para dos números y punto. El día que haga falta un tercer dato, el papelito deja de ser un papelito y **pasa a ser una ficha** — con su cajón, su número propio y sus casillas. Lo vas a ver en el paso 6.

3 Los pasos

3.1 Paso 1 · La tabla que aparece sola

Qué vamos a hacer

Declarar, en la ficha del trámite, la lista de documentos que pide. Y mirar el cajón que aparece.

Para entenderlo mejor

Es abrir el tercer cajón por primera vez. Tú declaras la relación en Java; el cajón —con sus papelitos— lo pone JPA.

El problema

En la base, la relación ya está prevista: db/migration crea **tres** tablas, no dos. La tercera es ésta:

```
create table tramite_documento (  
    tramite_id    bigint not null references tramite (id),  
    documento_id  bigint not null references documento (id),  
    primary key (tramite_id, documento_id)  
);
```

Dos columnas, y **la clave primaria son las dos juntas**. Eso es lo que impide que el mismo documento entre dos veces en el mismo trámite. Lo que falta es decirlo en Java.

La alternativa, y por qué no

- **Guardar los ids en un texto separado por comas** ("1,3,5"): cabe en una casilla, y a cambio pierdes el join, la integridad referencial y la posibilidad de preguntar «¿qué trámites piden la cédula?». Es la solución que parece más simple el primer día y la que más cara sale el segundo.
- **Crear tú la entidad intermedia** con dos @ManyToOne: funciona, es lo correcto cuando la relación tiene datos propios, y **hoy no los tiene**. Escribirla ahora es una clase de más. Es exactamente el paso 6.
- **@ManyToMany con @JoinTable**, que es lo de aquí: dos anotaciones, y la tabla del medio la gestiona JPA.

Se pega

En practica/src/main/java/cl/dgt/muchosamuchos/entities/Tramite.java, **entre los campos**:

```
@ManyToMany(fetch = FetchType.LAZY)  
@JoinTable(  
    name = "tramite_documento",  
    joinColumns = @JoinColumn(name = "tramite_id"),  
    inverseJoinColumns = @JoinColumn(name = "documento_id"))  
private Set<Documento> documentos = new LinkedHashSet<>();
```

Y al final de la clase, antes del toString(), el getter de la colección y los dos métodos que la tocan — uno detrás de otro:

```
public Set<Documento> getDocumentos() {  
    return documentos;  
}  
  
public void adjuntar(Documento documento) {  
    documentos.add(documento);  
}
```

```
public void quitar(Documento documento) {
    documentos.remove(documento);
}
```

Tres anotaciones y cada una dice una cosa:

- **@ManyToMany** — muchos de esto van con muchos de aquello. No hay columna que valga.
- **@JoinTable(name = ...)** — cómo se llama el cajón. Este lado, el que la lleva escrita, es **el dueño**: es quien mete y saca papelitos.
- **joinColumns / inverseJoinColumns** — cuál de las dos columnas del papelito es la mía y cuál es la del otro.

Y fetch = FetchType.LAZY, que en @ManyToMany **ya es el valor por defecto**. Se escribe igual: @ManyToOne es EAGER por defecto y @ManyToMany es LAZY, y nadie se acuerda de cuál es cuál. Escribirlo cuesta cinco palabras; suponerlo cuesta un incidente.

Lo que vas a ver

Después de llenar la demo del paso 1 —el guion PAS05.md trae el bloque entero— y reiniciar:

```
=== 1 · LA RELACIÓN · @ManyToMany y @JoinTable ===
5 documentos guardados
Hibernate:
    insert into tramite (fecha, rut, tipo) values (?, ?, ?)
Hibernate:
    insert into tramite_documento (tramite_id, documento_id) values (?, ?)
Hibernate:
    insert into tramite_documento (tramite_id, documento_id) values (?, ?)
Hibernate:
    insert into tramite_documento (tramite_id, documento_id) values (?, ?)
Hibernate:
    insert into tramite_documento (tramite_id, documento_id) values (?, ?)
3 trámites guardados · el 1 lleva 4 documentos
y CEDULA la piden los tres
tramite_documento · recién sembrada -> 9 filas
    tramite_id=1 documento_id=1
    tramite_id=1 documento_id=2
    tramite_id=1 documento_id=4
    tramite_id=1 documento_id=5
    tramite_id=2 documento_id=1
    tramite_id=2 documento_id=2
    tramite_id=2 documento_id=3
    tramite_id=3 documento_id=1
    tramite_id=3 documento_id=4
```

Un INSERT en tramite, y aparte cuatro en tramite_documento. En el Lab 05 la relación viajaba dentro del INSERT del trámite, en su columna. Aquí es una fila propia en otra tabla.

Y mira documento_id=1 —la cédula—: aparece con los tres trámites. **Eso es lo que una columna no podía guardar.**

Vas bien si...

Salen 9 filas en tramite_documento y el documento_id=1 aparece tres veces.

Si te atascas

1 · Schema-validation: missing table [tramite_documento]

El nombre del @JoinTable no coincide con el de la migración. Se escriben igual, letra por letra.

2 · missing column [tramite_id]

joinColumns es **tu** columna y inverseJoinColumns es la del otro. Si las cambias de sitio, Hibernate busca columnas que no existen.

3.2 Paso 2 · Meter y sacar papelitos

Qué vamos a hacer

Adjuntar un documento a un trámite, quitarlo, y mirar el cajón entre medio.

Para entenderlo mejor

La colección de Java y el cajón de papelitos son **la misma cosa vista dos veces**. Añadir a la colección es meter un papelito; quitar de la colección es sacarlo. Tú no escribes ni el insert ni el delete.

El problema

Lo que hay que entender aquí no es cómo se hace —es una línea— sino **qué pasa exactamente en la base**, y sobre todo **qué NO pasa**.

Se pega

En practica/src/main/java/cl/dgt/muchosamuchos/demos/DemosManyToMany.java, reemplazando el método agregarYQuitar() entero:

```
@Transactional
public void agregarYQuitar() {
    seccion(2, "AGREGAR Y QUITAR · los INSERT y DELETE de la intermedia");

    Trámite trámite = tramites.findById(trámiteInicioId).orElseThrow();
    Documento poder = documentos.findByCodigo("PODER").orElseThrow();
    System.out.println(" trámite " + trámite.getId() + " · " + trámite.getTipo()
        + " · lleva " + trámite.getDocumentos().size() + " documentos");

    System.out.println(" --- se ADJUNTA el poder ---");
    trámite.adjuntar(poder);
    tramites.flush();
    mirador.imprimirDelTrámite(trámiteInicioId, "tras adjuntar");

    System.out.println(" --- se QUITA el poder ---");
    trámite.quitar(poder);
    tramites.flush();
    mirador.imprimirDelTrámite(trámiteInicioId, "tras quitar");
}
```

Lo que vas a ver

```
=== 2 · AGREGAR Y QUITAR · los INSERT y DELETE de la intermedia ===
trámite 1 · Inicio de actividades · lleva 4 documentos
--- se ADJUNTA el poder ---
Hibernate:
    insert into tramite_documento (tramite_id, documento_id) values (?, ?)
tramite_documento · tras adjuntar -> 5 filas
--- se QUITA el poder ---
Hibernate:
    delete from tramite_documento where tramite_id=? and documento_id=?
tramite_documento · tras quitar -> 4 filas
```

Tres cosas, y las tres importan:

1. El SELECT de la colección llega **cuando la tocas** —`getDocumentos().size()`—, no antes. Es el mismo LAZY del Lab 05: `@ManyToMany` también es perezoso.
2. Adjuntar es **un insert de una fila**. Quitar es **un delete con las dos claves** en el where. Precisos, mínimos.
3. **No se borró el documento**. `quitar()` saca el papelito, no la ficha: el poder sigue en la tabla documento y sigue adjunto al trámite 2. Romper una relación y borrar una cosa son operaciones distintas, y en material tributario confundirlas sale caro.

Ese delete de ahí arriba lleva **las dos claves**. Acuérdate, porque en el paso 4 va a dejar de llevarlas.

Vas bien si...

Tras adjuntar hay 5 filas y tras quitar vuelven a ser 4. Y `select count(*) from documento` sigue dando 5.

Si te atascas

1 · No sale ningún insert.

Falta el `flush()`, o el método no es `@Transactional`. Sin transacción abierta, el cambio en la colección no llega a ninguna parte.

2 · Sale el insert pero no el delete.

`quitar()` usa `Set.remove()`, y un Set decide qué es «el mismo elemento» con `equals` y `hashCode`. Documento los trae escritos por su código; si los borras, `remove` no encuentra nada que quitar y no falla — simplemente no hace nada.

3.3 Paso 3 · El otro lado del cajón

Qué vamos a hacer

Leer la relación al revés: dado un documento, en qué trámites aparece.

Para entenderlo mejor

Es el mismo cajón de papelitos, leído por la otra columna. Antes buscabas todos los papelitos que empiezan por «trámite nº 1»; ahora, todos los que acaban en «documento nº 3».

El problema

La ficha del documento no lleva escrita ninguna lista de trámites, igual que la del contribuyente no llevaba la de los suyos en el Lab 05. Lo que necesita es una forma de decir «*mis trámites son los que me nombran en un papelito*».

La alternativa, y por qué no

- **Declarar otro @JoinTable en Documento:** compila, y es un error caro. Serían **dos dueños de la misma tabla** sin saber el uno del otro, y cada uno la escribiría a su manera.
- **mappedBy**, que es lo de aquí: «la verdad vive enfrente, en el campo documentos de Tramite». Este lado **no guarda nada**. Es exactamente el @OneToMany(mappedBy) del Lab 05, con otra anotación.

Se pega

En practica/src/main/java/cl/dgt/muchosamuchos/entities/Documento.java, **entre los campos:**

```
@ManyToMany(mappedBy = "documentos")
private Set<Tramite> tramites = new LinkedHashSet<>();
```

Y su getter, **antes del equals:**

```
public Set<Tramite> getTramites() {
    return tramites;
}
```

mappedBy = "documentos" nombra **el campo de la otra clase**, no la tabla ni la columna. Si te equivocas de nombre, la aplicación no arranca y te lo dice.

Lo que vas a ver

```
=== 3 · LADO ESPEJO · @ManyToMany(mappedBy) ===
documento: Cédula de identidad del representante
--- todavía NO se ha tocado la colección ---
Hibernate:
    select ...
    from tramite_documento t1_0
    join tramite t1_1 on t1_1.id=t1_0.tramite_id
    where t1_0.documento_id=?
aparece en 3 trámites:
    Tramite{id=1, tipo='Inicio de actividades', ...}
    Tramite{id=3, tipo='Término de giro', ...}
    Tramite{id=2, tipo='Cambio de representante legal', ...}
```

La misma tabla, leída al revés. Antes where tramite_id=?, ahora where documento_id=.

Y fíjate en el orden: 1, 3, 2. **No está prometido.** Un Set no tiene orden y ese select no lleva order by. Si el orden importa, se pide; si no lo pides, no lo tienes.

Vas bien si...

La cédula aparece en 3 trámites, y el SELECT de la colección sale **después** de la línea que dice «todavía NO se ha tocado».

Si te atascas

1 · `mappedBy` reference an unknown target entity property

El nombre entre comillas no es el del campo de `Tramite`. Es documentos, en plural.

2 · Sale una segunda tabla `documento_tramite` en la base.

Pusiste `@JoinTable` en los dos lados en vez de `mappedBy` en uno. Sólo uno manda.

3.4 Paso 4 · Set contra List, contado

Qué vamos a hacer

Hacer el cambio más pequeño posible —adjuntar un documento, y quitarlo— con un contador en marcha. Primero con `Set`, después con `List`.

Para entenderlo mejor

El archivista tiene dos formas de actualizar el cajón cuando le traes un documento nuevo:

- **Buscar dónde va y meter ese papelito.** Un movimiento.
- **Sacar TODOS los papelitos de ese trámite, tirarlos, y volver a escribirlos todos** — los que ya estaban y el nuevo. Tantos movimientos como documentos tenga el trámite, más uno.

El segundo suena absurdo, y es exactamente lo que hace Hibernate si declaras la colección como `List`.

El problema

Todo el mundo escribe `List`. Es el tipo que uno teclea sin pensar. Y no da error: funciona, los datos quedan bien, los tests pasan. Lo único que cambia es el número de sentencias, y ése no se ve si no se cuenta.

La alternativa, y por qué no

- `List<Documento>`: para Hibernate es una **bolsa** (bag) — admite repetidos y no tiene identidad por elemento. Como no puede saber *qué* fila cambió, hace lo único seguro: borrar la relación entera del trámite y reinsertarla completa.
- `Set<Documento>`, que es lo de aquí: tiene identidad por elemento —para eso `Documento` lleva `equals` y `hashCode` por su código—, así que cada cambio es la sentencia que le corresponde. Y encima dice la verdad sobre la tabla, cuya clave primaria ya prohíbe los repetidos.

Se pega

En `practica/src/main/java/cl/dgt/muchosamuchos/demos/DemosManyToMany.java`, reemplazando el método `setContraList()` entero:

```

@Transactional
public void setContraList() {
    seccion(4, "SET CONTRA LIST · el mismo cambio, medido");

    Tramite tramite = tramites.findById(tramiteInicioId).orElseThrow();
    Documento poder = documentos.findByCodigo("PODER").orElseThrow();
    System.out.println(" trámite " + tramite.getId() + " con "
        + tramite.getDocumentos().size() + " documentos ya cargados en memoria");

    System.out.println(" >>>>> EMPIEZA EL CONTEO · ADJUNTAR UN DOCUMENTO");
    contador.reiniciar();
    tramite.adjuntar(poder);
    tramites.flush();
    informe("ADJUNTAR UN DOCUMENTO");

    System.out.println(" >>>>> EMPIEZA EL CONTEO · QUITAR ESE MISMO DOCUMENTO");
    contador.reiniciar();
    tramite.quitar(poder);
    tramites.flush();
    informe("QUITAR ESE MISMO DOCUMENTO");
}

```

La carga queda **fuera** del conteo a propósito: el contador se reinicia justo antes del cambio. Lo que se mide es el cambio, no la lectura.

Lo que vas a ver

Con Set, que es como está:

```

>>>>> EMPIEZA EL CONTEO · ADJUNTAR UN DOCUMENTO
Hibernate:
    insert into tramite_documento (tramite_id, documento_id) values (?, ?)
<<<<< FIN DEL CONTEO · ADJUNTAR UN DOCUMENTO
    SENTENCIAS CONTRA LA BASE: 1

```

Ahora, **sin tocar la demo**, cambia el campo de Tramite a List<Documento> documentos = new ArrayList<>() —y su getter, y los dos import—, reinicia y mira el mismo tramo:

```

>>>>> EMPIEZA EL CONTEO · ADJUNTAR UN DOCUMENTO
Hibernate: delete from tramite_documento where tramite_id=?
Hibernate: insert into tramite_documento (tramite_id, documento_id) values (?, ?)
Hibernate: insert into tramite_documento (tramite_id, documento_id) values (?, ?)
Hibernate: insert into tramite_documento (tramite_id, documento_id) values (?, ?)
Hibernate: insert into tramite_documento (tramite_id, documento_id) values (?, ?)
Hibernate: insert into tramite_documento (tramite_id, documento_id) values (?, ?)
<<<<< FIN DEL CONTEO · ADJUNTAR UN DOCUMENTO
    SENTENCIAS CONTRA LA BASE: 6

```

El número, para el mismo cambio sobre un trámite que ya lleva cuatro documentos:

	adjuntar 1 documento	quitar 1 documento
Set<Documento>	1 sentencia	1 sentencia
List<Documento>	6 sentencias	5 sentencias

Y lo que hay que mirar de verdad es el primer delete:

```
delete from tramite_documento where tramite_id=?
```

Sin documento_id. No borra la fila que cambió: **borra la relación entera del trámite**, y después la reinserta completa. Con Set ese delete llevaba las dos claves; con List lleva una.

En un @ManyToMany, Set. Siempre. No es preferencia de estilo. Con 4 documentos son 5 sentencias de más; con 30 son 30. Y todas ellas escriben, no leen.

Vuelve a dejarlo en Set antes de seguir.

Vas bien si...

Con Set sale SENTENCIAS CONTRA LA BASE: 1 dos veces. Con List, 6 y 5.

Si te atascas

1 · Con List sale duplicate key value violates unique constraint.

Estás adjuntando un documento que el trámite ya tenía. Con Set ni se intentaría —un Set no admite repetidos—; con List sí se intenta, y la clave primaria de la tabla lo rechaza. Es el mismo defecto visto desde otro lado.

2 · El contador dice siempre 0.

Falta `generate_statistics: true` en el `application.yml`. Sin eso, `ContadorDeSentencias` devuelve cero y no falla.

3.5 Paso 5 · Un nombre de método que atraviesa el cajón

Qué vamos a hacer

Preguntar «¿qué trámites piden la cédula?» sin escribir SQL.

Para entenderlo mejor

Es pedirle al archivista que cruce los dos cajones **y el del medio**: buscar los papelitos que acaban en el número de la cédula, y traer las fichas de trámite que empiezan esos papelitos.

El problema

El código del documento no está en la tabla `tramite`. Está a dos tablas de distancia.

La alternativa, y por qué no

- **Tres consultas a mano:** el documento por su código, sus papelitos, y los trámites. Funciona, y son tres viajes donde basta uno.
- **@Query con JPQL** escrito a mano: control total, y hay que escribirlo.
- **Una consulta derivada que navega**, que es lo de aquí: `findByDocumentosCodigo` — «por la colección `documentos`, su campo `codigo`». Spring Data arma los dos `join`.

Se pega

En practica/src/main/java/cl/dgt/muchosamuchos/repositories/TramiteRepository.java:

```
package cl.dgt.muchosamuchos.repositories;

import cl.dgt.muchosamuchos.entities.Tramite;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface TramiteRepository extends JpaRepository<Tramite, Long> {

    List<Tramite> findByDocumentosCodigo(String codigo);
}
```

Lo que vas a ver

```
=== 5 · CONSULTA QUE NAVEGA · findByDocumentosCodigo() ===
Hibernate:
    select
        t1_0.id, t1_0.fecha, t1_0.rut, t1_0.tipo
    from
        tramite t1_0
    left join
        tramite_documento d1_0
        on t1_0.id=d1_0.tramite_id
    left join
        documento d1_1
        on d1_1.id=d1_0.documento_id
    where
        d1_1.codigo=?
trámites que piden PODER -> 1
trámites que piden CEDULA -> 3
```

Un solo SELECT con DOS join, y el de en medio es el cajón de los papelitos. En el Lab 05 salía uno; aquí salen dos porque hay una tabla más que atravesar. **Nadie los escribió**: salieron del nombre del método.

Tampoco lleva order by, así que el orden no está prometido. Si hace falta: findByDocumentosCodigoOrderById.

Vas bien si...

PODER devuelve 1 trámite y CEDULA devuelve 3, con un solo select cada uno.

Si te atascas

1 · No property 'documentosCodigo' found

El camino no existe. Se desambigua con guion bajo: findByDocumentos_Codigo.

2 · Devuelve 0.

El código que pasas no coincide. Están en mayúsculas: CEDULA, PODER.

3.6 Paso 6 · Cuando @ManyToMany deja de servir

Qué vamos a hacer

Hacerle a la tabla intermedia una pregunta que no puede contestar, y dibujar en qué se convierte. **Aquí no se implementa nada.**

Para entenderlo mejor

Vuelve al papelito. Dice «*trámite n° 1 — documento n° 3*» y no cabe una palabra más. El día que la oficina quiera saber **cuándo** se adjuntó ese documento, o **quién** lo trajo, el papelito se queda corto: hay que ascenderlo a ficha, con su propio número y sus propias casillas.

El problema

@JoinTable manda una tabla de exactamente dos columnas. No hay ningún sitio donde poner una fecha. Y no es un caso raro: en el SII, casi toda relación de este tipo acaba queriendo una fecha.

La alternativa, y por qué no

- **Añadir la columna a mano en la migración:** la tabla la tendría, y JPA no la vería, no la escribiría y la dejaría a null. Peor que no tenerla.
- **Convertir la intermedia en entidad,** que es lo que toca: una clase Adjunto con @Id propio y dos @ManyToOne — al trámite y al documento— más sus columnas. Y en Tramite, un @OneToMany(mappedBy = "tramite").

Lo bueno: **no hay nada nuevo que aprender.** Un @ManyToMany partido en dos son dos @ManyToOne, que es el Lab 05 hecho dos veces. Lo que se gana es una tabla con nombre y con sitio. Lo que se pierde es la comodidad de tramite.adjuntar(documento).

Se pega

En practica/src/main/java/cl/dgt/muchosamuchos/demos/DemosManyToMany.java, reemplazando el método cuandoDejaDeServir() entero:

```
public void cuandoDejaDeServir() {
    seccion(6, "CUÁNDO @ManyToMany DEJA DE SERVIR");

    System.out.println(" la pregunta: ¿en qué fecha se adjuntó CEDULA al trámite "
        + tramiteInicioId + ", y quién la subió?");
    System.out.println(" columnas de tramite_documento -> " + mirador.columnas());
    System.out.println(" la respuesta no está, y no cabe: @ManyToMany manda una tabla
        ↪ de dos claves");
    System.out.println();
    System.out.println(" EN QUÉ SE CONVIERTE (no se implementa hoy):");
    System.out.println(" @Entity @Table(name = \"adjunto\")");
    System.out.println(" class Adjunto {");
    System.out.println("     @Id @GeneratedValue Long id");
    System.out.println("     @ManyToOne(LAZY) Tramite tramite");
```

```

System.out.println("      @ManyToOne(LAZY)      Documento documento");
System.out.println("      LocalDate      fechaAdjunto      <-- el dato
↳ propio");
System.out.println("      String      subidoPor      <-- y este");
System.out.println("    }");
System.out.println("    y en Tramite: @OneToMany(mappedBy = \"tramite\")
↳ Set<Adjunto> adjuntos");
System.out.println();
System.out.println("    la tabla intermedia deja de ser un @JoinTable y pasa a ser
↳ una @Entity.");
System.out.println("    Y entonces esto ya no es muchos-a-muchos: son dos
↳ @ManyToOne, que es el lab 05 dos veces.");
}

```

Lo que vas a ver

=== 6 · CUÁNDO @ManyToOne DEJA DE SERVIR ===
 la pregunta: ¿en qué fecha se adjuntó CEDULA al trámite 1, y quién la subió?
 columnas de tramite_documento -> [tramite_id, documento_id]
 la respuesta no está, y no cabe: @ManyToOne manda una tabla de dos claves

EN QUÉ SE CONVIERTE (no se implementa hoy):

```

@Entity @Table(name = "adjunto")
class Adjunto {
    @Id @GeneratedValue Long id
    @ManyToOne(LAZY)    Tramite tramite
    @ManyToOne(LAZY)    Documento documento
    LocalDate           fechaAdjunto      <-- el dato propio
    String              subidoPor         <-- y este
}

```

y en Tramite: @OneToMany(mappedBy = "tramite") Set<Adjunto> adjuntos

La máquina lo dice sola: columnas de tramite_documento -> [tramite_id, documento_id]. Dos. No hay hueco para una fecha.

Vas bien si...

La lista de columnas sale con exactamente dos nombres.

Si te atascas

1 · La consulta a information_schema devuelve vacío.

Estás mirando otra base. El puerto de practica/ es el **55447**; el de solucion/, el 55448.

4 Lo que aprendiste

1 · Un muchos-a-muchos es una tabla, no una columna.

Y esa tabla no tiene clase, ni @Entity, ni repositorio: la declara @JoinTable, y JPA la llena y la vacía. El lado que lleva el @JoinTable es **el dueño**; el otro usa mappedBy y no guarda nada — exactamente igual que en el Lab 05.

2 · El tipo de la colección tiene precio, y lo mediste.

1 sentencia con Set, 6 con List, para el mismo cambio. Con `List`, Hibernate borra la relación entera del trámite y la reinserta completa, porque una bolsa no tiene identidad por elemento. En un `@ManyToMany`, `Set`.

3 · @ManyToMany es LAZY por defecto, y aun así se escribe.

`@ManyToOne` es `EAGER` por defecto y `@ManyToMany` es `LAZY`. Son valores distintos para anotaciones que se parecen. Escribir `fetch = FetchType.LAZY` aunque sea redundante hace que quien lea la clase lo sepa sin ir a buscar la especificación.

4 · @ManyToMany se acaba en cuanto la relación tiene algo que decir de sí misma.

Una fecha, una firma, un estado. Entonces la intermedia pasa a ser una entidad con dos `@ManyToOne`, y no es nada nuevo: es el Lab 05, dos veces.

5 · Y el N+1 del Lab 06 aplica aquí igual, y peor.

Listar 200 trámites y tocar los documentos de cada uno son **201 consultas**, como en el Lab 06. Peor por dos motivos: cada una de esas 200 atraviesa **dos** tablas en vez de una, y el otro lado hace lo mismo al revés. Las tres salidas del Lab 06 —`join fetch`, `@EntityGraph` y la proyección— valen aquí sin cambiarles una coma.

5 Para profundizar

- **Adjunta dos veces el mismo documento** al mismo trámite con `Set` y luego con `List`. Con uno no pasa nada; con el otro, la clave primaria de la tabla te para. ¿Cuál de los dos te está protegiendo?
- **Cuenta las sentencias con un trámite de 20 documentos** en vez de 4, con `List`. Comprueba si la cuenta es la que esperas.
- **Añade `findByDocumentosCodigoIn(List<String> codigos)`** y mira qué `join` sale.
- **Dibuja en papel** la entidad `Adjunto` del paso 6, con sus dos `@ManyToOne`. Después mira el `Tramite` del Lab 05 y compáralos: son la misma clase escrita dos veces.
- **Pon `@ManyToMany(fetch = FetchType.EAGER)`** y cuenta las consultas al listar los 3 trámites. Esa cuenta es el Lab 06.

6 Antes de cerrar

Párala con Ctrl+C.

```
./mvnw clean
```

Lo que te llevas:

Un muchos-a-muchos es una tabla de pares que nadie escribe. El lado con `@JoinTable` manda; el otro usa `mappedBy` y no guarda nada. Y la colección va en `Set`, porque con `List` cada cambio borra la relación entera y la rehace.

Lo que queda pendiente, y abre el Lab 06: hoy contaste 6 sentencias donde había 1, con cuatro documentos. Con doscientos trámites en una pantalla, la cuenta que sale es **201 consultas contra 1**. En el Lab 06 se mide, se ve y se arregla de tres formas distintas.